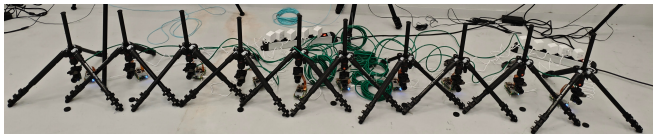
Synchronised Multi-camera Video Capture with a Distributed Fleet of 10 Raspberry Pi Cameras: Architecture, Synchronisation, and Evaluation Methodology

Mattéo Pêcher
Univ. Grenoble Alpes / Grenoble INP — Ensimag (M1 MoSIG)
Internship hosted at Inria Grenoble, Morpheo team
matteo.pecher@grenoble-inp.org
Supervised by: Julien Pansiot (Inria Grenoble)

I understand what plagiarism entails and I declare that this report is my own, original work.
Matteo Pecher, 05/06/2026



Abstract

We present KINOFLEX, a low-cost synchronised capture rig built from 80 Raspberry Pi 5 single-board computers, each fitted with a Sony IMX296 global-shutter camera, plus a Raspberry Pi Pico microcontroller that broadcasts a shared external-trigger pulse at 60 fps. We work here on stage 1 of KINOFLEX, composed of 10 nodes.

The contribution is twofold. First, we introduce the full system: a hardware-triggered acquisition pipeline with on-node delta+zstd compression, an MQTT-based (Message Queuing Telemetry Transport) orchestration plane, a PYSIDE6 supervisor GUI, parallel SCP export, and an Ansible-based deployment layer for the 10-node fleet. Second, we propose the evaluation methodology built around it: a three-tier tests suite (unit, deployment, chaos), and a synchronisation-comparison protocol that benchmarks different synchronisation protocols.

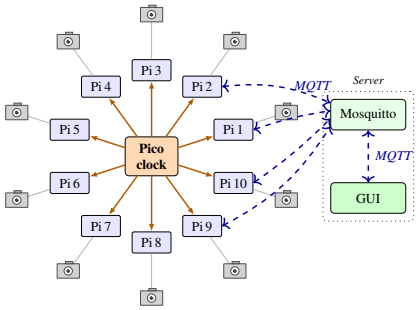


Figure 1: Physical disposition of the KINOFLEX fleet.

1 Introduction

1.1 Context and Motivation

Volumetric video — the capture, reconstruction and free-viewpoint rendering of dynamic three-dimensional scenes — has become a central research challenge at the intersection of computer vision, computer graphics and high-performance computing. Recent advances on dynamic neural radiance fields [Pumarola *et al.*, 2021; Mildenhall *et al.*, 2020] and on volumetric 4D capture platforms have shown that the field is moving from offline curiosities towards real-time, free-viewpoint media. Concrete applications now span immersive virtual / augmented reality, broadcast sports, biomechanics and cultural heritage [Kinovis Project, 2015; Morpheo Team, Inria, 2018].

Classical 4D capture platforms rely on dense arrays of calibrated cameras that simultaneously record a scene from multiple viewpoints. The captured images are then processed offline to reconstruct a dynamic geometry-and-texture model that can be rendered from arbitrary viewpoints. For instance, the KINOVIS platform at Inria Grenoble employs 65 colour cameras with resolutions ranging from 5 to 25 Mp over a 10×10m capture area [Morpheo Team, Inria, 2018], while the University of Surrey rig [Hilton *et al.*, 2011] extends a similar approach beyond controlled studio environments. Although these dedicated systems deliver high-quality reconstructions, their cost, complexity, and physical footprint remain prohibitive for many research groups.

Despite the remarkable performance of KINOVIS, the platform still exhibits several limitations, notably its high cost and limited portability, which constrain broader accessibility and adoption. This motivates the development of a low-cost, open-source alternative such as KINOFLEX. By reducing hardware and deployment barriers, KINOFLEX aims not only to expand the range of practical applications for 4D capture systems but also to facilitate easier deployment and wider accessibility.

1.2 Problem Statement

So to make KINOFLEX a reality we need to solve 4 intertwined engineering problems :

1. **Synchronisation.** All cameras must expose within the same physical instant; otherwise the multi-view recon-

struction, which assumes every view shares a single time instant may fail dramatically.

2. **Storage.** 80 cameras at 1456×1088@60 fps represent an aggregate raw rate above 22 GB/s; per-node, the IMX296 stream alone reaches roughly 280 MB/s.
3. **Deployment.** An 80-node Linux fleet has to be reproducibly re-imageable; ad-hoc `ssh` loops are not an option.
4. **Operability.** A single operator must be able to start, stop, monitor, export and triage the rig.

1.3 Research Questions and Hypotheses

The work is organised around three research questions. **(RQ1)** *What level of synchronisation accuracy can be achieved on a fleet of commodity Pi 5 boards equipped with hardware-triggered global-shutter sensors, and how much do the standard alternatives (free-run, NTP,* LIBCAMERA *master/slave) lose against it?* Our working hypothesis is that the hardware trigger reaches the sub-microsecond regime, but this needs to be quantified with an independent ground truth. [1] **(RQ2)** *Can lossless real-time compression keep the per-node NVMe write rate within the sustained capability of a DRAM-less drive over a single-lane PCIe 3.0 link?* **(RQ3)** *Is the orchestration layer (deployment, supervision, export, regression tests) robust enough to be operated by a single person?*

The output of KINOFLEX is a synchronised, compressed and exported set of per-camera image streams that can be fed directly into a regular 4D reconstruction pipeline; coupling it end-to-end with a dynamic-NeRF or 4D Gaussian renderer is left as future work. As we work on stage one of KINOFLEX we limit ourselves here to a setup of 10 Pis.

1.4 Contributions and Outline

This report describes the full pipeline as it stands at the end of the internship. Section 2 positions the work with respect to existing multi-camera platforms, synchronisation protocols and lossless video codecs. Section 3 details our acquisition stack: the Pico clock generator, the Pi-side capture/writer/metrics process model and the on-disk delta+zstd format. Section 4 covers the server-side supervisor: MQTT service, reactive PYSIDE6 state model, page auto-discovery, session database and parallel SCP exporter. Section 5 introduces the synchronisation-comparison protocol — free-run, NTP, LIBCAMERA master/slave and the hardware trigger — and the 23-LED probe panel used as ground truth. Section 6 describes the three-tier test suite that pins the behaviour of the system. Section 7 concludes and outlines what we would tackle next.

2 State of the Art

Before describing our own design, we situate it against the three bodies of prior work that each of the engineering problems above draws on: the multi-camera platforms that define the application, the synchronisation protocols we benchmark against, and the lossless codecs that make on-node storage feasible.

2.1 Multi-Camera Capture Platforms

Dedicated volumetric studios represent the gold standard for 4D acquisition. KINOVIS [Morpheo Team, Inria, 2018], funded as an *Équipement d'Excellence*, host a platform at Inria Grenoble. Similar large-scale rigs exist at Kyoto University and at the University of Surrey, where Hilton et al. [2011] survey the challenges of taking 4D acquisition *beyond the studio*. Commercial products such as 4DViews and 3dMD provide turnkey solutions but remain limited to small capture volumes.

Camera *synchronisation* is a prerequisite for every such platform. Global shutter sensors are strongly preferred over rolling shutter sensors because they expose all pixels simultaneously, avoiding the per-row time offsets that produce "jello" artefacts under motion. The IMX296 sensor used in our rig provides a dedicated XTRIG external-trigger input: integration starts on the falling edge, ends 14.26 μs after the rising edge, and the frame is then read out [Raspberry Pi Ltd., 2023]. Hardware trigger lines remain the most reliable synchronisation method, achieving alignment errors well below a frame period [Zhou *et al.*, 2024].

2.2 Network Synchronisation Protocols

Two software-only approaches are relevant here.[2] NTP, the *de facto* Internet time protocol designed by Mills [1991], maintains accuracies of a few milliseconds on the public Internet and around 0.1 ms on fast LANs. This is one to two orders of magnitude too coarse to align two 60 fps frames within their exposure window: at the short shutter times relevant to fast motion (down to a few tens of microseconds), a sub-millisecond offset already exceeds the exposure itself. A more camera-specific option is the master/slave synchronisation built into LIBCAMERA: one camera is designated the *server* (master) and periodically broadcasts its frame timing, while the others act as *clients* (slaves) that nudge their frame durations to converge on the server's cadence. Because it aligns frame boundaries entirely in software, without any external trigger or timestamping hardware, its accuracy stays in the same sub-millisecond range as NTP on a fast LAN — still well above the exposure window for fast motion.

Subframe-level vision-based methods have been proposed to recover the temporal offset *a posteriori* [Zhou *et al.*, 2024]; LED-based probes, in particular, are a classical practical tool to measure the residual offset between unsynchronised cameras up to subframe precision by recording a known binary pattern in the scene [Shrestha *et al.*, 2007]. We re-use the latter idea in Section 5 as a ground-truth oracle, not as a synchronisation mechanism in itself.

[1] We expect it to be two to three orders of magnitude tighter than software-only schemes such as NTP or LIBCAMERA's master/slave synchronisation; this is exactly what the evaluation of Section 5 is designed to quantify.

[2] We also considered PTP (IEEE 1588) [IEEE, 2020] for hardware-timestamped synchronisation, but on the Raspberry Pi 5 hardware timestamping is exposed only on the wired Ethernet interface and not over Wi-Fi; adopting it would have constrained the whole fleet to a wired network, so we did not pursue it.

2.3 Lossless Compression for High-Rate Acquisition

At 1456×1088@60 fps in 8-bit RGB the per-node stream is around 95 Mpixel/s, i.e. ∼280 MB/s before file-system overhead. The single-lane PCIe 3.0 link of the Pi 5 and the DRAM-less NVMe drives we use cannot sustain that throughput once application and FTL (Flash Translation Layer) overhead are added, so real-time compression is mandatory in the acquisition loop.

FFV1 [Niedermayer *et al.*, 2021] is a strong archival lossless codec but its software implementations are too heavy at our target rate on ARM. JPEG-LS (LOCO-I) [Weinberger *et al.*, 2000] in true lossless is also too slow for our hardware. Our choice is therefore a real-time lossless delta pipeline backed by Zstandard [Collet and Kucherawy, 2021]: each frame is encoded as the pixel-wise difference from the previous frame in the block, then entropy-coded on-node before the disk write. Zstandard is designed for real-time scenarios at zlib-level ratios and significantly higher throughput [Collet and Kucherawy, 2021], which makes it a natural fit on the 4-core ARM Cortex-A76 featured in the Raspberry Pi 5.

3 Acquisition Stack

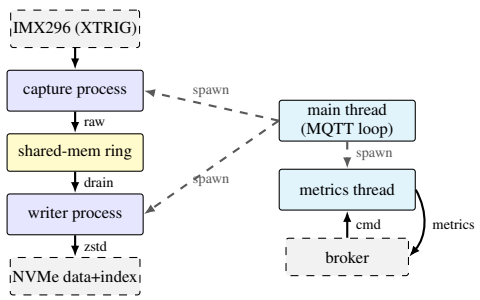
With the prior art in place, we now turn to our own design, starting at the sensor end of the pipeline. This section addresses the hardware side of **RQ1** (the trigger generator and its distribution) and **RQ2** (real-time on-node lossless compression), working outward from the camera trigger to the bytes on disk.

3.1 Hardware Synchronisation: the Pico Clock

A Raspberry Pi Pico (RP2040) generates the master 60 Hz trigger pulse on a GPIO line. The Pico's Programmable I/O (PIO) state machine runs at 10 MHz, giving a 100 ns resolution on both the high (active) and low (inactive) phases of the trigger; the PIO program is fed by DMA, so the CPU is not involved in pulse timing and jitter is essentially independent of the firmware workload.

The trigger output is fanned out to all IMX296 sensors via a star wiring topology, as shown on Figure 1. Because the IMX296 starts integration on the falling edge of XTRIG and ends it 14.26 μs after the rising edge [Raspberry Pi Ltd., 2023], the exposure time is controlled by the *low* phase of the trigger plus this fixed offset.

3.2 Pi Client: Process Model



Each Pi runs a python program as a systemd service. The main thread carries the MQTT event loop and spawns three workers via `multiprocessing`: a **capture** process that drives the camera in external-trigger mode and writes raw buffers into shared memory; a **writer** process that drains the queue, groups frames into 60-frame blocks, computes per-frame deltas and saves zstd-compressed blocks to an NVMe; and a **metrics** thread that publishes one consolidated JSON per interval.

3.3 Why `multiprocessing` rather than threading?

The two CPU-bound paths — libcamera readout and zstd compression — would otherwise serialise on Python's GIL. Splitting them into separate processes lets capture run on one core and the writer on another, and removes the most common cause of frame drops in earlier prototypes (zstd backpressure starving the capture loop). The shared-memory ring buffer between the two processes is generously over-sized.

3.4 Acquisition Handshake

Hardware synchronisation only buys correctness if the start/stop protocol respects it. Our handshake is:

1. Server stops the Pico clock.
2. Server broadcasts `start session`; each Pi enters `acquiring` after a 2 s sensor settle plus a ghost-frame drain.
3. Once *every* selected Pi is in `acquiring`, the server restarts the Pico clock — the first trigger pulse is now seen by all sensors at the same time.
4. On stop, the symmetric protocol applies: stop clock, wait for the Pico to acknowledge, then a 5 s grace, broadcast `stop`, wait for every Pi to return to `ready`, then restart the clock to free-run between acquisitions.

3.5 On-disk Format

Each session is written as two companion files. The first one is `Index.bin`, it opens with a 32-byte header (frame width, height, channel count, fps and block size) followed by one entry per 60-frame block. The other one is `Data.bin`, it's the concatenation of zstd-compressed delta blocks. Per-file SHA-256 sidecars are produced at stop time and verified after each SCP transfer. Observed compression ratios on typical scenes range from 3× to 6×. Because the codec is a frame-to-frame delta, the ratio is governed mostly by how much the image changes between consecutive frames: static, low-texture scenes with a fixed background and little motion yield near-zero deltas and compress best, whereas fast motion, scene complexity and sensor noise enlarge the deltas and push the ratio towards the lower bound. Even at that lower bound the effective NVMe write rate drops to a sustainable ∼50 MB/s — our positive answer to **RQ2**: lossless on-node compression keeps the per-node write rate within the sustained envelope of a DRAM-less PCIe 3.0 x1 NVMe drive.

4 Server-side Supervisor

Having a stack that captures and compresses correctly is only half the problem; the rig also has to be deployable and driveable by one person. This section addresses **RQ3**: making the rig operable by a single person via an idempotent deployment pipeline, a reactive supervisor, a SQLite session catalogue and parallel SCP export.

The server runs on a single workstation that also hosts the Mosquitto MQTT broker [OASIS, 2019; Light, 2017]. We chose MQTT over HTTP/gRPC because its publish/subscribe model naturally fits a 1-to-10 broadcast pattern (easily scalable to 80); the broker decouples server and clients, lets clients reconnect independently and avoids 10 long-lived TCP connections on the server side. Mosquitto in particular is lightweight enough to coexist with the GUI on the same machine.

4.1 Deployment via Ansible

Before any of the orchestration can run, the 10-node fleet has to be brought to an identical, known state. The fleet is provisioned by an idempotent Ansible playbook [Red Hat, Inc., 2024] that installs the OS packages, pins `libcamera` 0.7.0,[3] mounts the NVMe, applies the NVMe-stall mitigations (disable PCIe ASPM, re-enable Host Memory Buffer), installs the camera trigger-mode service, and deploys and enables the client code. Because the playbook is idempotent, a re-run only touches what has drifted, so the same command both provisions a fresh node and repairs an existing one.

4.2 Reactive State and Page Auto-discovery

With every node provisioned identically, the operator drives the whole fleet from a single window. The supervisor is a PySide6 application centred on a single `AppState` object that aggregates `PiState` children, a `ClockState` and a `ServerState`; all of them are `QObject`s that emit Qt signals on property change. `MqttService` is the *only* writer: it parses incoming messages (status string, metrics JSON, frames JSON, snapshot blobs framed with a per-request token, FS-RPC responses) and mutates the state.

The state is surfaced to the operator through a set of *pages*. The supervisor exposes 14 such pages, including the Dashboard (preview grid, clock controls, range slider, export), per-metric grids (CPU, RAM, disk, temperatures, network, drops), a Pipeline-Health page (queue depth, per-block writer time, compression ratio), a Sessions page (recordings table, per-Pi export status, scrub previews, fleet-wide "Clear SSDs"), a Timestamps page (inter-frame deltas from the metrics stream), an Event Log (persistent notification history) and a Tests page that drives the `pytest` suite from the GUI. Pages live in `server/pages/` and are *auto-discovered* at startup by scanning for `BasePage` subclasses with a `PAGE_ORDER` attribute; adding a page is a single-file operation, removing one is a single-file deletion. Each page subscribes to the state signals it cares about and redraws reactively; user actions flow back through `broadcast()` or `send_to_pi()`.

This single-writer / many-readers shape is essential. With possibly up to 80 Pis publishing metrics every second and 14 pages each watching a different subset of the state, anything more permissive would race. Concretely, every Pi metric is parsed exactly once into the matching `PiState` field, and Qt's signal/slot mechanism then notifies only the widgets that actually depend on that field — a grid widget showing the disk-fill chart of pi 12 does not redraw when pi 12's CPU temperature changes, even though both come from the same JSON payload.

4.3 Session Database and Parallel Export

Acquired sessions are catalogued in a SQLite database. Per-Pi frame counts, queued cleans for offline Pis (replayed when they reconnect) and slices of the JSONL event log + `journalctl` + `dmesg` are stored alongside the session metadata, so that a session can be debugged offline weeks later without the live fleet. The data files themselves stay on the Pis until the operator triggers an export.

The exporter uses a `ThreadPoolExecutor` of SCP workers to pull `data.bin` and `index.bin` from every Pi in parallel; the `.sha256` sidecars are pulled in the same batch and verified before the session is considered complete. A second path drives a Pi-side range-slicer so that the operator can export only the sub-range of interest from each Pi — the slicer pipelines SCP and decode across Pis, swaps the libcamera BGR bytes back to RGB and writes a single `frames.raw` plus a `meta.json` that records the pixel layout.

5 Synchronisation Evaluation Plan

Now that we have a fully functional stack that captures, compresses and exports synchronised streams, we still have to prove that the synchronisation claim actually holds. This section addresses **RQ1** head-on: *how good is the hardware trigger in practice, and how much do we actually gain over a purely networked solution (free-run, NTP,* LIBCAMERA *master/slave)?* We describe here the evaluation protocol that we designed; the numerical results are not yet available and will be presented at the defence.

5.1 Why an Optical Ground Truth at All?

A naive evaluation would compare per-camera `SensorTimestamp` values across the fleet and report the spread. Even if it is useful for day-to-day operations, it cannot validate the hardware trigger because `SensorTimestamp` is itself derived from the Pi's own system timer. Two cameras seeing slightly different XVS rising edges will, by design, latch slightly different sensor timestamps; the only way to verify that those timestamps reflect a real synchronisation guarantee is to bring an external optical clock into the scene that none of the four conditions can influence. This is the role of the LED probe.

5.2 Conditions Compared

We compare four synchronisation regimes on the same fleet and the same scene:

[3] Version 0.7.1 deadlocks in external-trigger mode; we opened a thread on the libcamera GitHub reporting the bug, the maintainers reproduced it and a fix is in progress. We pin 0.7.0 in the meantime.

Free-run. No common time base. Each sensor uses its on-chip oscillator at the nominal 60 fps. Establishes the worst-case baseline.

NTP. All Pis are slaved to a local `chrony` server on the workstation [Mills, 1991]. The expected accuracy on the LAN is in the sub-millisecond range — orders of magnitude coarser than the sub-microsecond hardware trigger.

Master/slave (libcamera). Same fleet, synchronised purely in software with LIBCAMERA's camera-synchronisation algorithm: one Pi acts as the server and broadcasts its frame timing, the others as clients that adjust their frame durations to match. No external trigger or special hardware is involved, so this is the most favourable software-only condition we can offer; its expected accuracy is in the sub-millisecond range, comparable to NTP.

Hardware (Pico). The proposed star-wired Pico trigger described in Section 3. The expected accuracy is dominated by wire-length differences and is well below 1 μs on our setup.

5.3 LED-Panel Ground Truth

Software-only timestamps cannot, by construction, validate a hardware-trigger claim — they would only measure the very same clock domain we want to audit. We therefore use an external optical probe inspired by classical LED-based offset estimators [Shrestha *et al.*, 2007].

The probe (shown on the left of Figure 2) is a panel of **23 LEDs** driven by a *second* Raspberry Pi Pico running a deterministic PIO program. Rather than the more classical binary-counter encoding, the probe uses a *cycling chase* pattern : exactly one LED is lit at any instant, and the lit position advances by one slot every $\tau_{\text{led}} = 1\,\mu$s, so the full cycle through the 23 LEDs takes 23 μs. The camera exposure for this evaluation is set to $T_{\text{exp}} = 20\,\mu$s — slightly shorter than one full cycle of the chase.

The decoding then becomes purely geometric. During an exposure window of 20 μs, the chase covers about 20 of the 23 LED slots; the remaining two or three are never lit during that frame. Two regimes therefore appear in the recorded images:

- **Best case (exposure aligned on an LED boundary):** exactly 3 consecutive LEDs are perfectly dark and the other 20 are fully lit. The identity of the central dark LED localises the exposure within the 23 μs cycle with 1 μs resolution.
- **Worst case (exposure midway through an LED slot):** two adjacent LEDs are perfectly dark, flanked by two LEDs that are *partially* lit (the chase visited each of them only during a fraction of the exposure window). The brightness ratio between those two partial LEDs encodes the sub-μs offset of the exposure midpoint with respect to the LED boundaries — modulo the full acquisition transfer function (LED rise/fall time, sensor ADC, analogue gain and gamma), which must be characterised and calibrated out before the ratio can be read as a linear time offset.

This gives sub-microsecond precision in principle, limited in practice only by the sensor's photon shot noise and the linearity of the LED response over a 1 μs window. Across the four conditions of Section 5.2 the relevant metric is the spread of Δ across the nine cameras for the *same* hardware frame; a Pico-triggered fleet should exhibit that spread tightly below 1 μs, whereas the NTP and free-run conditions should be wide enough that the dark slot itself differs from camera to camera.

The probe Pico is intentionally *not* the same as the trigger Pico — it provides an *independent* clock domain, so a constant offset between the two Picos can be subtracted out without privileging the trigger condition. For each of the four conditions above, every camera records the panel; we extract the LED state from each frame and compare them across cameras.

5.4 Sweeping the LED Step Time

The previous subsection described the probe at a single step time ($\tau_{\text{led}} = 1\,\mu$s), the one chosen to obtain sub μs accuracy (Figure 2). The actual evaluation sweeps τ_{led} across *seven* step times: one frame period (≈ 16.67 ms, the chase advances one slot per frame — a pure frame-level synchronisation test, this way we can easily check that we don't drop frames), then 100 ms, 10 ms, 1 ms, 100 μs, 10 μs and 1 μs. Each campaign records the same scene under each of the four conditions at one step time, so the full matrix is $4 \times 7 = 28$ runs.

The crucial point is that, *except* for the free-run condition, none of the methods should accumulate drift over the duration of one campaign: NTP, LIBCAMERA master/slave and the hardware trigger all converge towards a common time base, so the spread observed across the nine cameras at a given step time is a direct estimate of the *jitter* of that method at that scale, uncontaminated by drift. As τ_{led} drops below the jitter floor of a given method, the dark-LED position stops carrying useful information (it varies camera-to-camera within one frame) and we can mark that scale as the resolution limit for that method. The seven step times therefore form a logarithmic ladder that brackets each method's jitter to within a decade.

Free-run is the only condition expected to drift *within* a campaign, because each sensor's on-chip oscillator runs at a slightly different rate; the recovered LED position should then drift linearly with time, and a linear regression on that drift gives a direct measurement of the inter-sensor oscillator mismatch.

5.5 Orders of Magnitude

Condition	Measured jitter
Free-run	drift
NTP	~ 1 ms
Master/slave	~ 1 ms
Pico	$< 1\,\mu$s

Table 1: Per-camera synchronisation jitter on our setup measured with the 23-LED panel.

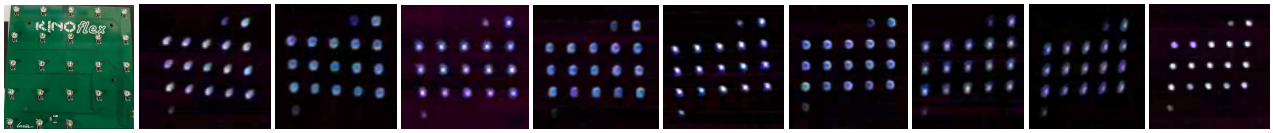


Figure 2: The *same* hardware-triggered frame as recorded simultaneously by nine cameras under the Pico trigger, with the LED chase advancing one slot per microsecond ($\tau_{\text{led}} = 1\ \mu$s). All nine cameras resolve the identical lit/dark LED pattern. The stability of the pattern across the fleet therefore bounds the inter-camera exposure offset below the 1 μs LED step, confirming the sub-microsecond accuracy of the external hardware trigger.

6 Tests and Validation

The test suite is the second half of our answer to **RQ3**: it is what keeps the rig safe.

A system of this size only stays correct if regressions are caught immediately. The test suite is split into three tiers that mirror the three things that can go wrong — a single component, the orchestration, or the rig as a whole — with roughly 150, 20 and 17 tests respectively.

Unit tests (∼150 tests, no fleet required) pin the small contracts in isolation: the on-disk format round-trip, Pico firmware/bridge constant parity, MQTT message dispatch, the FS-RPC sandbox, alert thresholds, the acquisition-handshake order and session-database CRUD. **Deployment tests** (∼20 tests) drive the actual fleet over MQTT — timed acquisitions that must reach zero drops with matching per-Pi frame counts, range exports whose frame counts and `frames.raw` sizes must match, and a check that the live status graph follows the documented state machine. **Chaos tests** (∼17 tests, deliberately push the system into bad states — stop without start, rapid start/stop cycles, orchestrator crash mid-acquisition, garbage payloads, ten back-to-back 30 s acquisitions, and a Pi rebooted mid-flight — and assert that it recovers to a known state without leaking file descriptors or shared memory.

Beyond catching regressions, this infrastructure surfaced two non-trivial hardware/driver bugs that no unit test could have predicted. The first was a recurring *30 s NVMe write stall* on the DRAM-less drives under sustained load, traced to a disabled Host Memory Buffer combined with PCIe ASPM L1 exit latency; the playbook now re-enables HMB and disables ASPM, and — as a second line of defence — the capture→writer shared-memory ring buffer is pre-allocated with roughly 2400 frame slots (∼40 s at 60 fps), large enough to ride out such a stall without dropping a single frame. The second was a `libcamera` 0.7.1 SIGSEGV in external-trigger mode worked around by pinning 0.7.0 (see the deployment footnote in Section 4).

7 Conclusion

We have built the stage 1 of KINOFLEX, a 10-node, hardware-triggered 4D acquisition rig running on commodity Raspberry Pi 5 hardware. The full stack is operational: a Pico clock generator and its MQTT bridge; a multi-process Pi client that captures, delta+zstd compresses and publishes consolidated metrics; a reactive PYSIDE6 supervisor with auto-discovered pages, a SQLite session database, parallel SCP export and a Pi-side range slicer; an idempotent Ansible deployment pipeline; and a three-tier test suite that pins the behaviour of the system end-to-end.

Mapped onto the three research questions, the rig delivers on all three. For synchronisation **(RQ1)** it provides a hardware trigger validated against an *independent* optical ground truth rather than the fleet's own clocks, reaching the sub-microsecond regime of Table 1 — two to three orders of magnitude tighter than the software-only NTP and LIBCAMERA master/slave baselines. For storage **(RQ2)** the real-time delta+zstd codec keeps every node's write rate comfortably within the envelope of a DRAM-less PCIe 3.0 x1 NVMe drive. For operability **(RQ3)** the reactive supervisor, session catalogue, parallel export, Ansible deployment and three-tier test suite together let a single person provision, drive and triage the whole fleet.

8 Future Work

Several extensions would naturally follow from the current state of the rig. First, the most immediate step is to scale from the 10-node development fleet to the full 80-node target and run the synchronisation campaign of Section 5 at the production scale. Second, untethering the nodes by adding battery packs and a Wake-on-LAN remote-power scheme would remove the last cabling constraint and make ad-hoc, outside-the-studio deployments practical. Third, replacing the wired Pico distribution with custom Wi-Fi synchronisation boards — each carrying a disciplined local oscillator phase-locked to a broadcast beacon — would preserve the sub-microsecond accuracy of the hardware trigger while eliminating the star-wiring harness entirely. Finally, the deployment layer could be hardened further by shipping pre-baked OS images instead of running the Ansible playbook from scratch, and by distributing the server-side stack as a single compiled binary rather than a Python virtual-environment checkout.

Acknowledgements

This work was supported by French government funding managed by the National Research Agency under the "Investments for the Future" program (PIA) grant ANR-21-ESRE-0030 (CONTINUUM) and the under the "France 2030" program, grant "ANR-24-RRII-0002" operated by the "Programme Inria Quadrant" (PIQ). I thank Julien Pansiot for his supervision and for hosting the project, and the M1 MoSIG teaching staff at Univ. Grenoble Alpes / Grenoble INP for the framing of the research internship.

References

[Collet and Kucherawy, 2021] Yann Collet and Murray Kucherawy. Zstandard compression and the `application/zstd` media type. *IETF RFC 8878*, pages 1–52, 2021. https://www.rfc-editor.org/info/rfc8478/.

[Hilton *et al.*, 2011] Adrian Hilton, Jean-Yves Guillemaut, Joe Kilner, Oliver Grau, and Graham Thomas. 3D-TV production from conventional cameras for sports broadcast. *IEEE Transactions on Broadcasting*, 57(2):462–476, 2011. https://ieeexplore.ieee.org/abstract/document/5756462.

[IEEE, 2020] IEEE. IEEE 1588-2019 — standard for a precision clock synchronization protocol for networked measurement and control systems. IEEE Std 1588-2019, 2020. https://ieeexplore.ieee.org/document/9120376.

[Kinovis Project, 2015] Kinovis Project. Kinovis – 4D Modeling Platform. https://kinovis.inria.fr, 2015. Inria Grenoble, Equipex funded by Investissement d'Avenir.

[Light, 2017] Roger A. Light. Mosquitto: server and client implementation of the MQTT protocol, 2017. https://www.researchgate.net/publication/317210457_Mosquitto_server_and_client_implementation_of_the_MQTT_protocol.

[Mildenhall *et al.*, 2020] Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. NeRF: Representing scenes as neural radiance fields for view synthesis. In *European Conference on Computer Vision (ECCV)*, pages 405–421, 2020. https://dl.acm.org/doi/abs/10.1145/3503250.

[Mills, 1991] David L. Mills. Internet time synchronization: The network time protocol. *IEEE Transactions on Communications*, 39(10):1482–1493, 1991. https://ieeexplore.ieee.org/abstract/document/103043.

[Morpheo Team, Inria, 2018] Morpheo Team, Inria. Activity report 2018: Morpheo — 4d modeling of moving bodies. Technical report, Inria Grenoble Rhône-Alpes, 2018. https://radar.inria.fr/rapportsactivite/RA2018/morpheo/morpheo.pdf.

[Niedermayer *et al.*, 2021] Michael Niedermayer, Moritz Bunkus, and Dave Richardson. FFV1 video coding format version 0, 1, and 3. *IETF RFC 9043*, pages 1–72, 2021. https://ftp.st.ryukoku.ac.jp/pub/internet/rfc/rfc9043.pdf.

[OASIS, 2019] OASIS. MQTT Version 5.0 — OASIS standard. OASIS Open, 2019. https://docs.oasis-open.org/mqtt/mqtt/v5.0/os/mqtt-v5.0-os.pdf.

[Pumarola *et al.*, 2021] Albert Pumarola, Enric Corona, Gerard Pons-Moll, and Francesc Moreno-Noguer. D-NeRF: Neural radiance fields for dynamic scenes. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 10318–10327, 2021. https://openaccess.thecvf.com/content/CVPR2021/papers/Pumarola_D-NeRF_Neural_Radiance_Fields_for_Dynamic_Scenes_CVPR_2021_paper.pdf.

[Raspberry Pi Ltd., 2023] Raspberry Pi Ltd. Global shutter camera — IMX296 external trigger specification. Raspberry Pi Forums and IMX296 datasheet, 2023. https://forums.raspberrypi.com/viewtopic.php?t=349260.

[Red Hat, Inc., 2024] Red Hat, Inc. Ansible: Simple IT automation. https://www.ansible.com, 2024. Accessed 2026-05-21.

[Shrestha *et al.*, 2007] Prabhakar Shrestha, Mauro Barbieri, and Hans Weda. Synchronization of multi-camera video recordings based on audio. In *Proceedings of the 15th ACM International Conference on Multimedia (MM '07)*, pages 545–548. ACM, 2007. Companion of the LED-based subframe synchronisation method used as ground truth in this report. https://dl.acm.org/doi/10.1145/1291233.1291367.

[Weinberger *et al.*, 2000] Marcelo J. Weinberger, Gadiel Seroussi, and Guillermo Sapiro. The LOCO-I lossless image compression algorithm: Principles and standardization into JPEG-LS. *IEEE Transactions on Image Processing*, 9(8):1309–1324, 2000. https://ieeexplore.ieee.org/abstract/document/855427.

[Zhou *et al.*, 2024] Xiaoshi Zhou, Yanran Dai, Haidong Qin, Shunran Qiu, Xueyang Liu, Yujie Dai, Jing Li, and Tao Yang. Subframe-level synchronization in multi-camera systems using time-calibrated video. *Sensors*, 24(21):6975, 2024. https://www.mdpi.com/1424-8220/24/21/6975.

A GUI Gallery

This appendix collects full-window screenshots of every page exposed by the KINOFLEX supervisor GUI, in the order they appear in the left navigation rail. Pages are auto-discovered from `server/pages/` and everything redraws reactively as MQTT telemetry arrives.



Figure 3: Selection page that allows to choose the synch method as well as the pis we want to monitor.

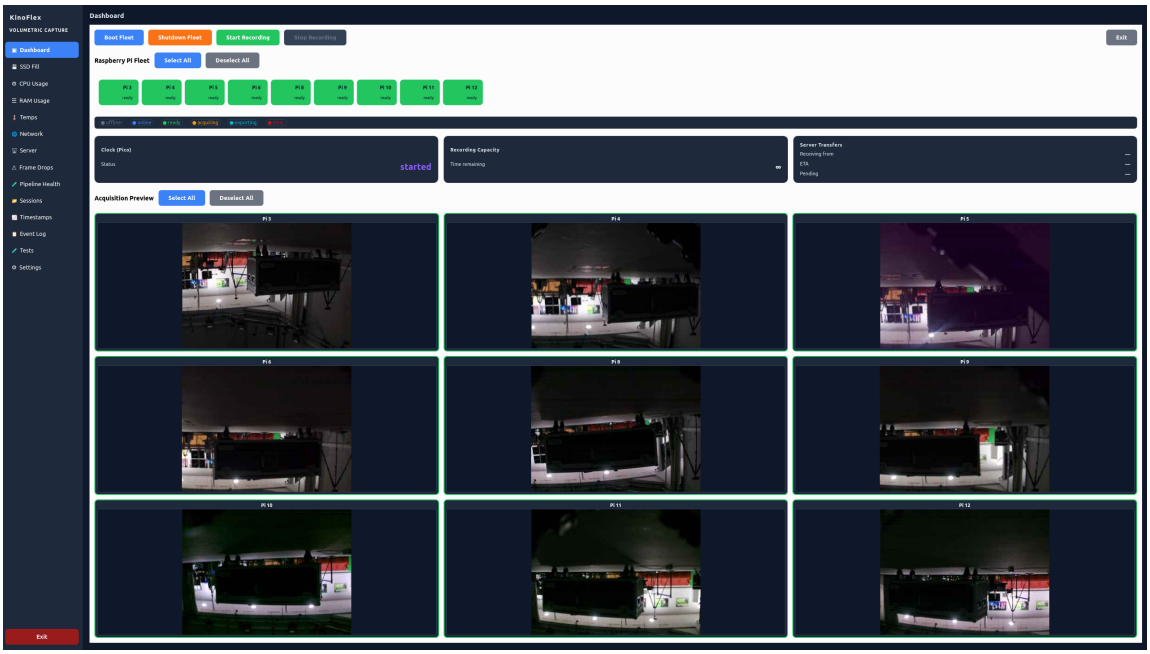


Figure 4: Dashboard page: the per-Pi live-preview grid, the clock controls, and the record/export range slider.

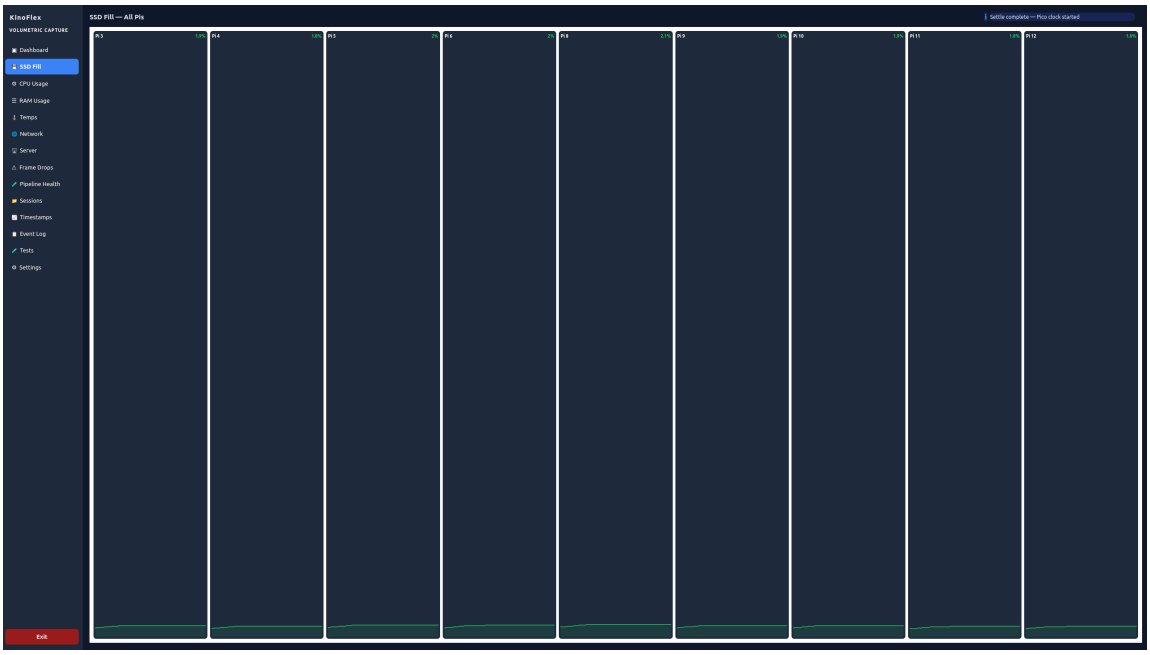


Figure 5: SSD Fill page: per-Pi disk-fill.

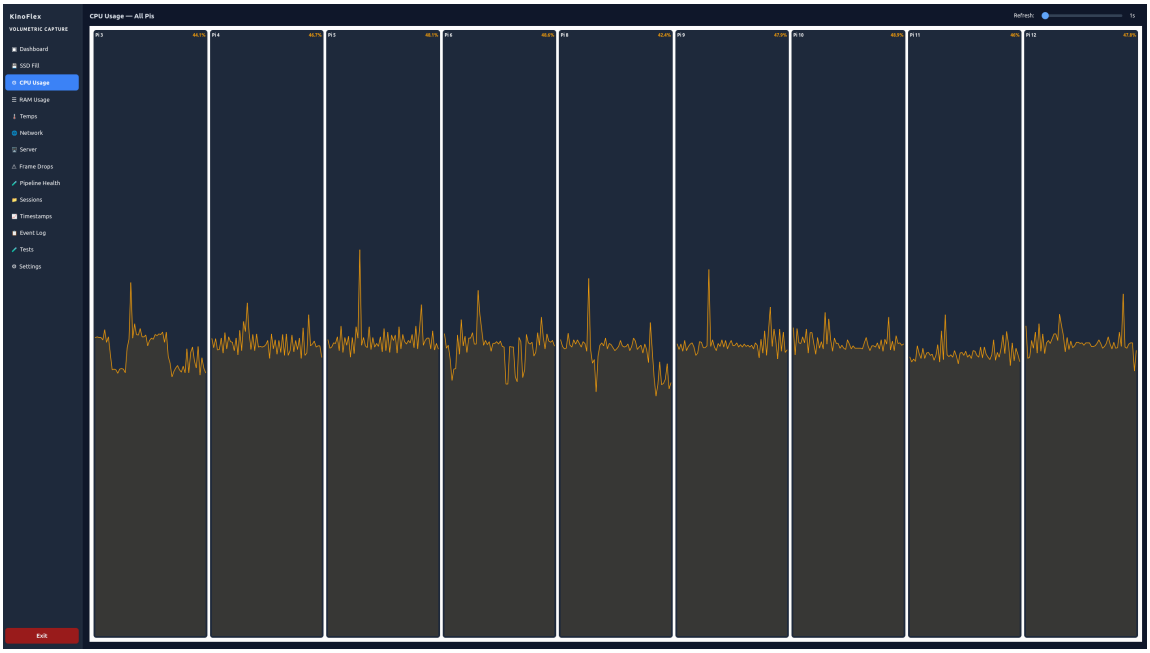


Figure 6: CPU Usage page: per-Pi CPU-utilisation.

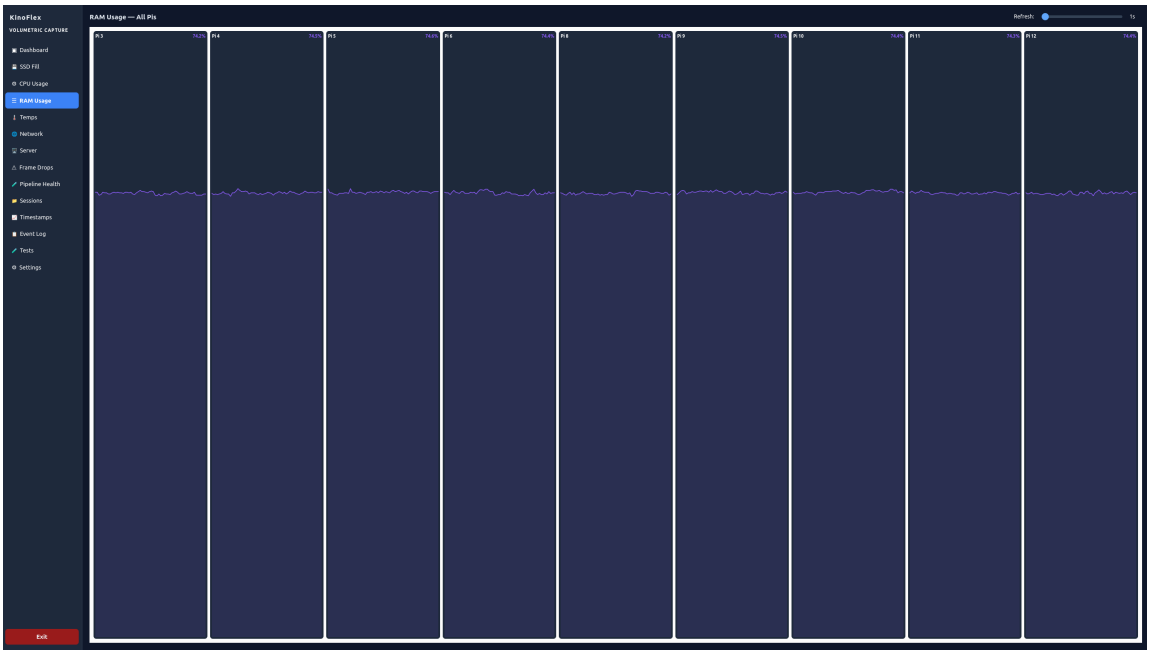


Figure 7: RAM Usage page: per-Pi memory-utilisation.

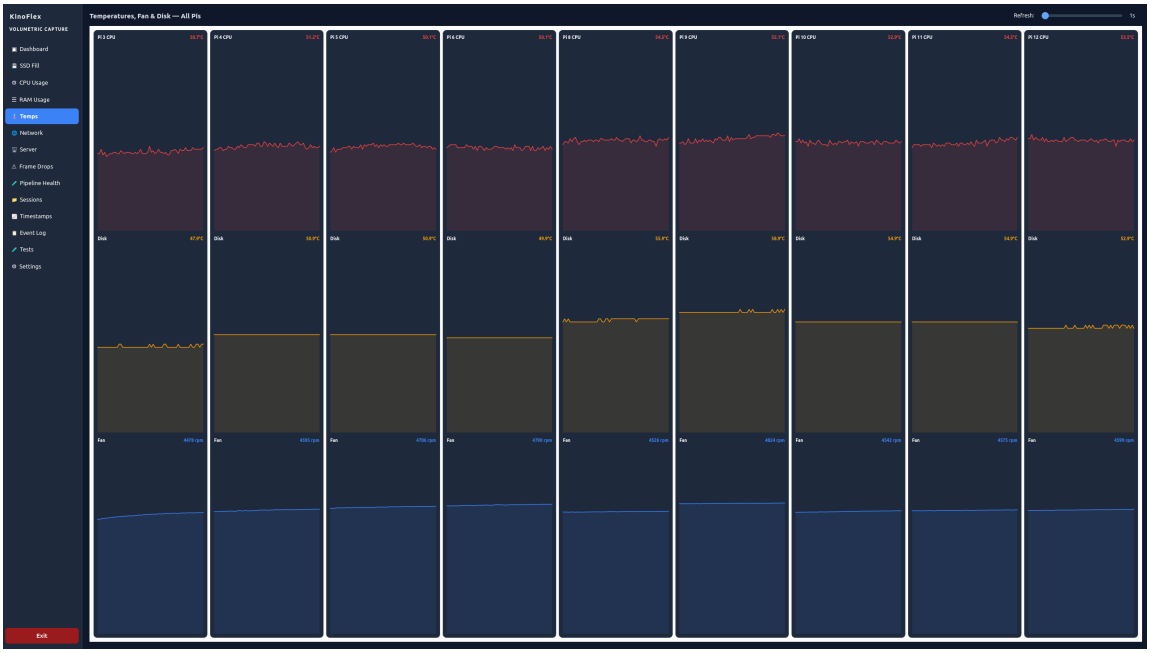


Figure 8: Temps page: CPU temperature, disk temperature, and fan RPM per Pi.

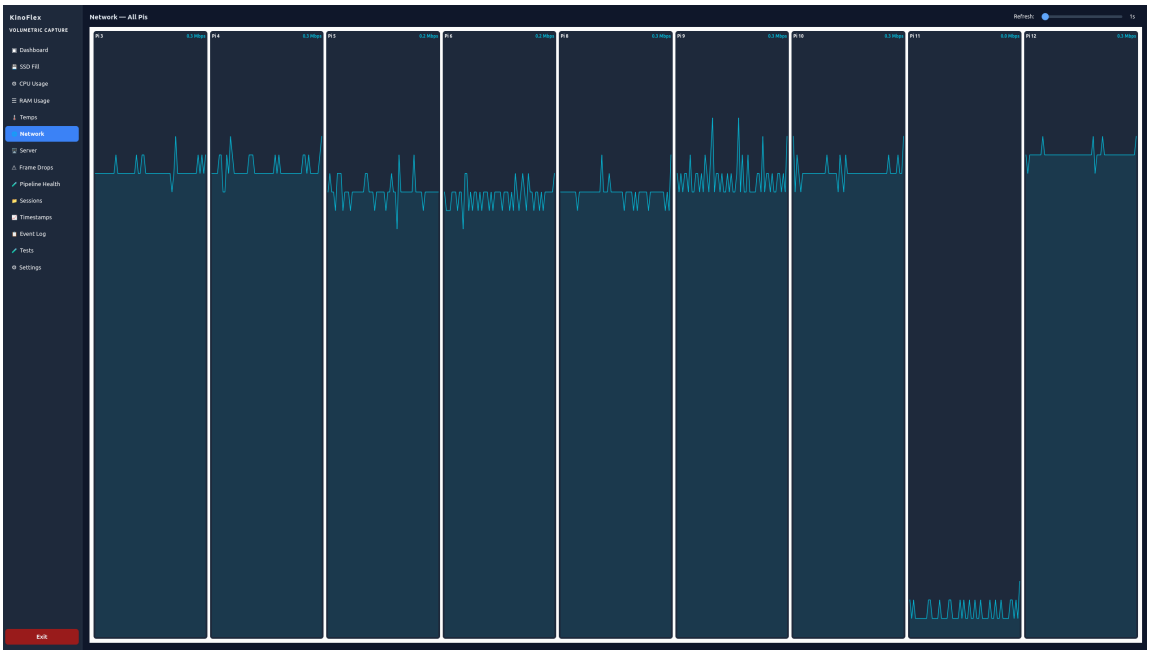


Figure 9: Network page: per-Pi network-throughput.

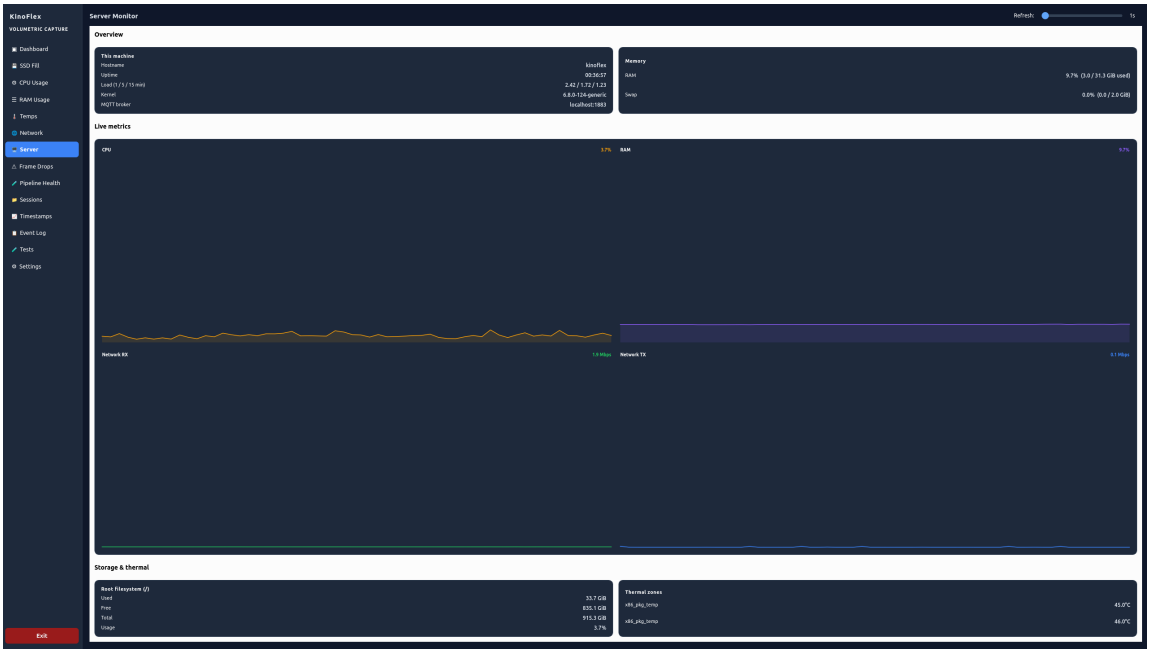


Figure 10: Server page: CPU, RAM, disk, network, and temperatures of the supervisor host itself.

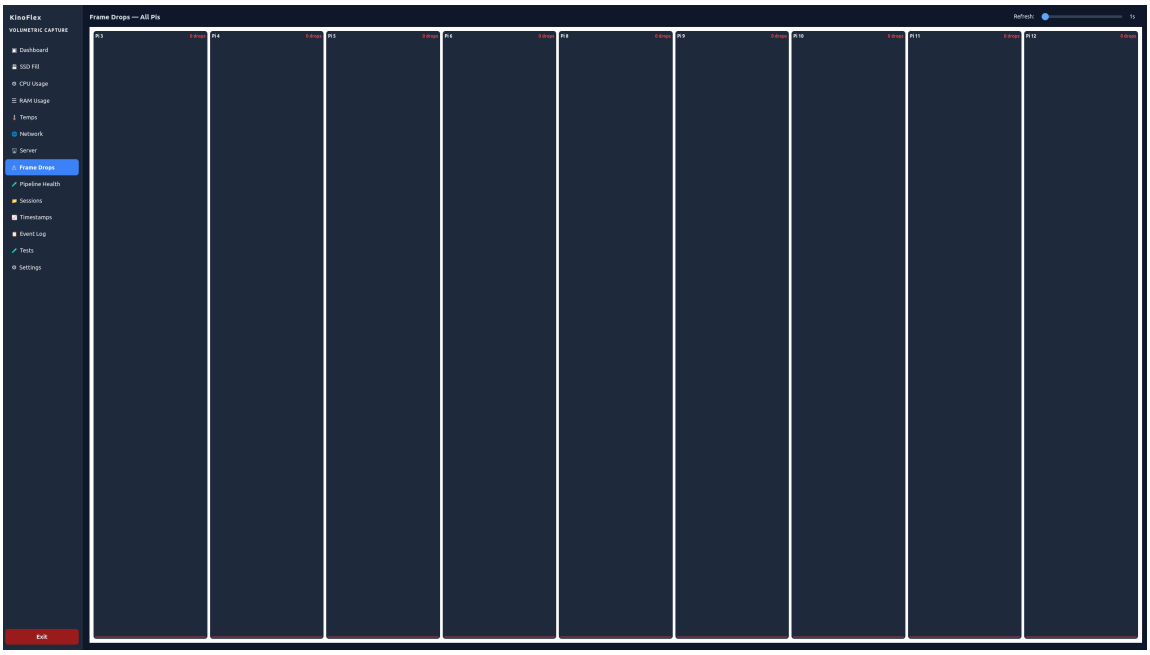


Figure 11: Frame Drops page: cumulative dropped-frame count per Pi.

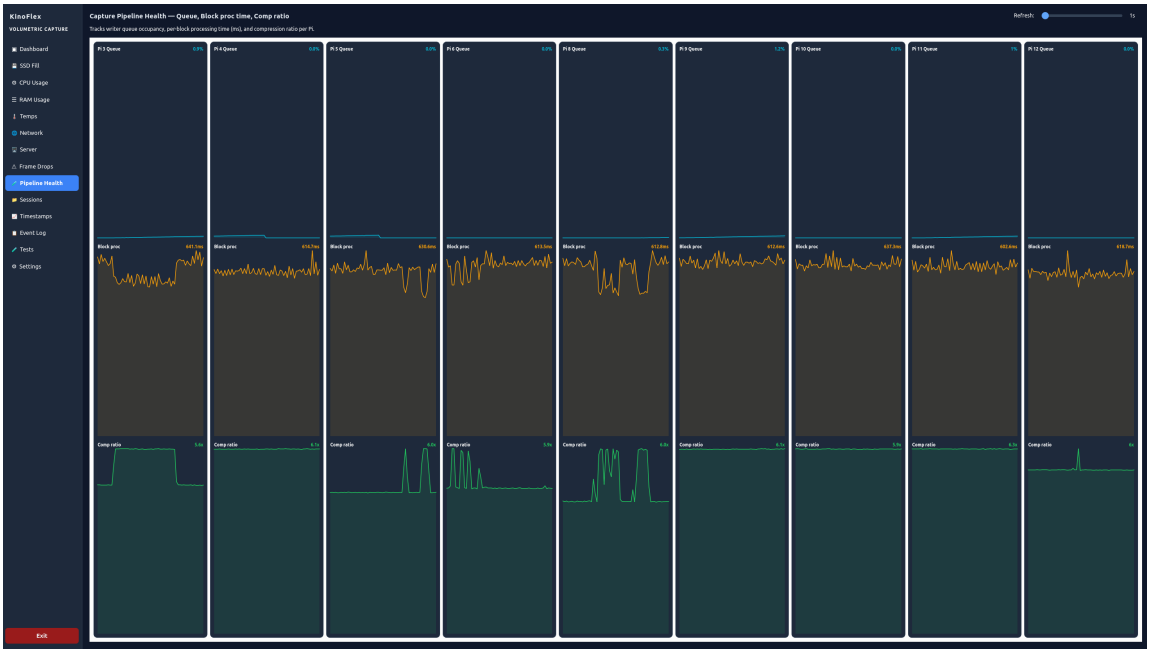


Figure 12: Pipeline Health page: queue-depth percentage, per-block writer time, and compression ratio.

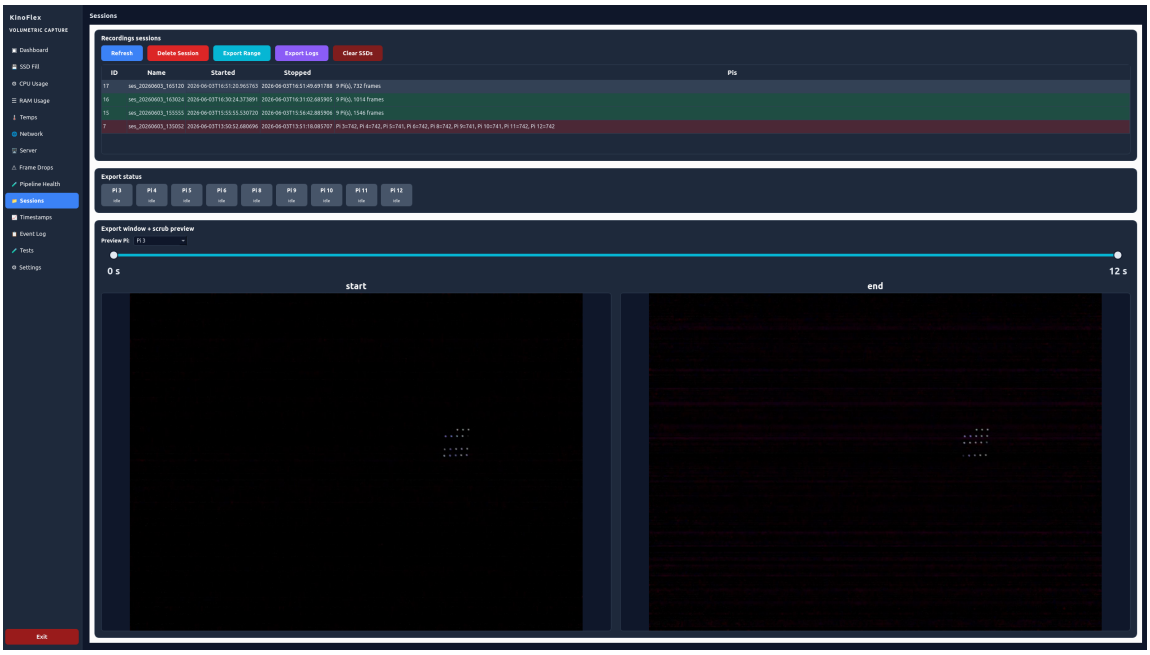


Figure 13: Sessions page: recordings table, per-Pi export status, scrub previews, and the fleet-wide “Clear SSDs” control.

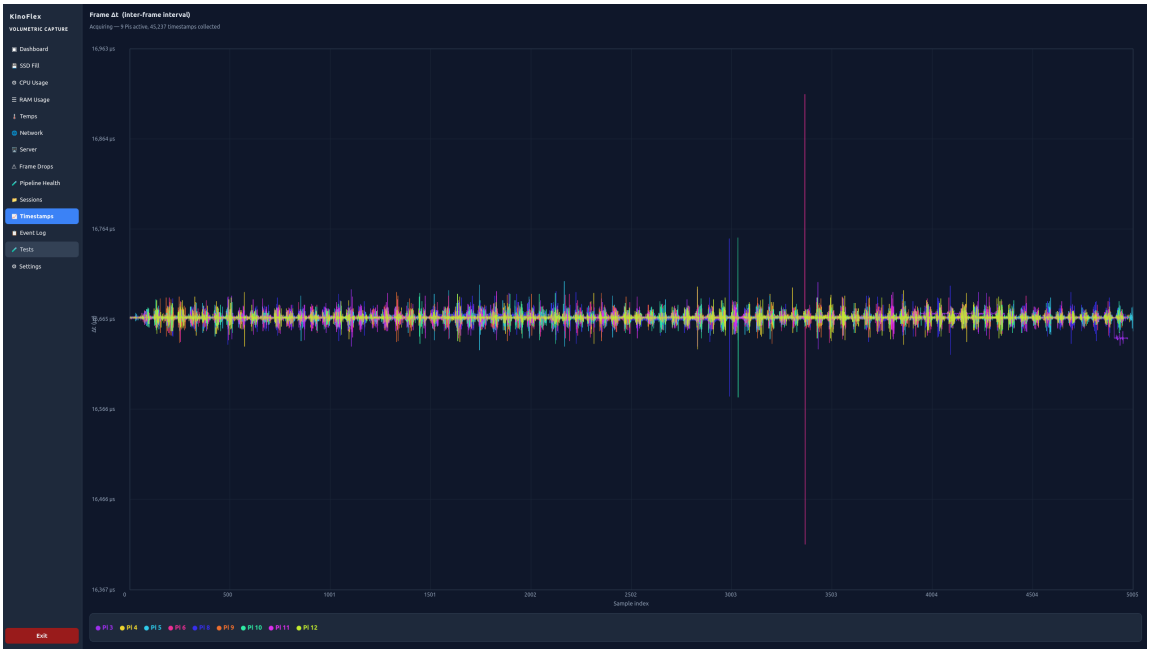


Figure 14: Timestamps page: live inter-frame delta plots derived from the metrics stream.



Figure 15: Event Log page: persistent, scrollable notification history.



Figure 16: Tests page: in-GUI launcher for the `pytest` suites.

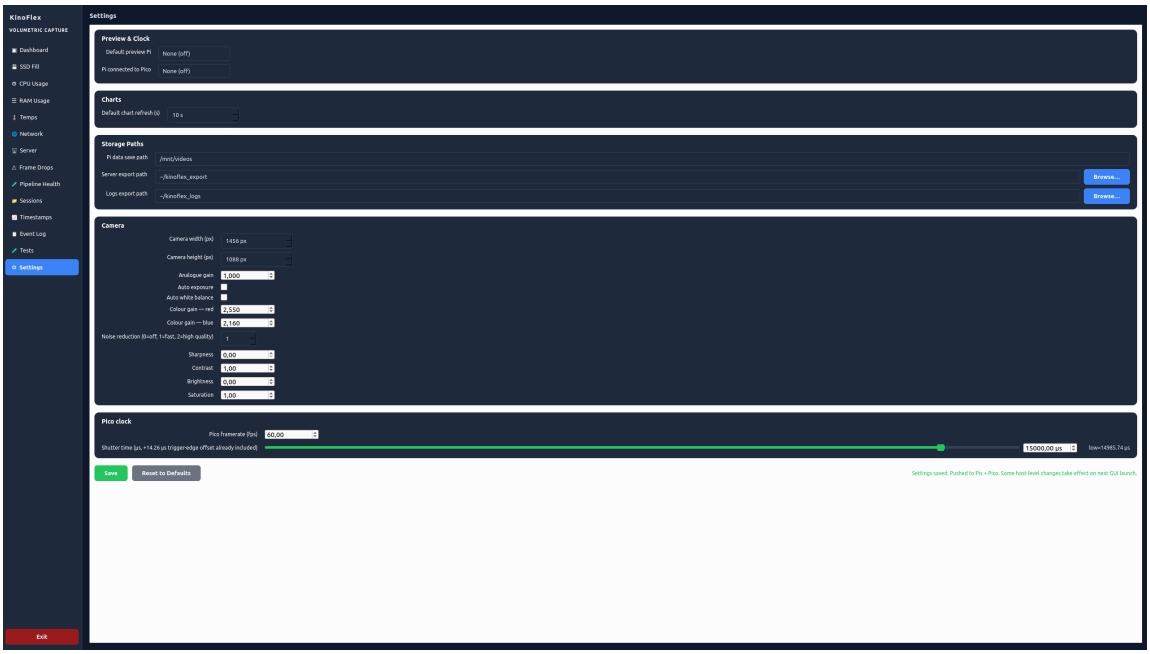


Figure 17: Settings page: storage paths, camera controls, and the Pico fps + exposure slider.